

Question 1 (Sensor Monitoring System)

Link:

https://github.com/helen751/Formative1-C_Programming/tree/main/question1_water_monitoring

❖ Technical Explanation

For my technical implementation, I created three functions.

The first function is `water_quality()`. This function is responsible for calculating the water-quality index. The formula was already given, so I used it to calculate the temperature deviation, the turbidity penalty, and the final water-quality index. **I used `fabsf()`** because I am working with floating-point numbers. Temperature and turbidity can be floating-point numbers, so `fabsf()` allows me to calculate the absolute temperature deviation without removing the decimal part.

The second function is `print_report()`. This function prints the temperature, turbidity, calculated water-quality index, and water-quality status. I used `if`, `else if`, and `else` statements to do the given checks and classify the water quality as Good, Warning, or Critical.

I also used console colors to make the results more readable. Since this is currently a C program running in a console. When it is changed into an embedded-system program. They could be changed into LCD display colors, LED colors, or whatever output is being used.

The third function is `main()`. This is where I called the other functions and run the main part of the program. I did not want to feed the program normal data only, so I used a loop that runs ten(10) times to generate controlled random numbers. The temperature values are from 0 to 79, and the turbidity values are from 0 to 99. During every loop, the program generates the two sensor readings, calls `water_quality()` to calculate the index, and then calls `print_report()` to display the result.

I also used `sleep(2)` to delay the program for two seconds and imitate real-life sensor readings. I would have to use `delay(2000)` instead in arduino.

a. Real-world application

This program could be used in a water-quality monitoring system, whereby a temperature sensor and a water-turbidity measuring sensor are used to get the sensor readings. The sensor readings are then fed into the microcontroller, which interprets them. When the water quality is bad, the system could raise an alarm using a buzzer. Instead of the `printf()` statements, we could have a screen, such as an LCD screen, to show the quality of the water. It could also raise an alarm when the water is bad.

I think this is the same logic used in the drinking-water machine we have in school, because the machine also filters and measures the quality of the water. When the quality of the water is bad, there is an LED that turns red on it. When the water is safe to drink, there is an LED that shows green. Therefore, this is a good real-world application of this program.

C is suitable for this application because it can work with microcontrollers and hardware components such as sensors, LCD screens, alarms, and LEDs. It can process the sensor readings and control what the embedded system should do based on the calculation results.

b. Error analysis

An example of a syntax error is forgetting a semicolon: *`float index = water_quality()`*

This is considered a syntax error because the statement does not follow the rules of the C language. The compiler would not be able to compile the program until the semicolon is added.

A semantic error that I experienced was using `abs()` for the temperature-deviation calculation:

```
float temperatureDeviation = abs(temperature - 25);
```

I was being warned by my IDE and compiler, so I had to do some research. I found that `abs()` is used for integer values. Since I am working with temperature values and it could have decimals, I had to use `fabsf()`:

```
float temperatureDeviation = fabsf(temperature - 25.0f);
```

This is a semantic error because the program may still compile and run, but it would not work correctly with the floating-point value. It could remove the decimal part and give an inaccurate result meaning an incorrect logic implementation.

c. Compilation lifecycle

My C source code must go through different stages before it becomes an executable program:

1. **Preprocessing**

The input is my original C source file. At this stage, the preprocessor handles the header files that I included, such as `stdio.h`, `stdlib.h`, `math.h`, `time.h`, and `unistd.h`. It also handles the color definitions created using `#define`. The output is the expanded source code.

2. **Compilation**

The input is the expanded source code from the preprocessing stage. The compiler checks the code for errors and translates it into assembly instructions. The output is assembly code.

3. **Assembly**

The input is the assembly code. The assembler changes it into machine code. Output is an object file.

4. **Linking**

The input is the object file and the required C libraries. The linker joins them together and connects the library functions I used, such as `printf()`, `rand()`, `time()`, `fabsf()`, and `sleep()`.

5. **The output is now the final executable program.**

I was able to run the executable program. The loop inside my `main()` function runs `ten(10)` times and gives me different readings for testing. It calculates and displays a new result every two seconds. This allowed me to see whether the program could classify different readings as Good, Warning, or Critical. I used this approach to imitate a real water monitoring system that reads data from sensors every 2 seconds.

d. Sample Output

```
===== WATER SENSOR MONITORING REPORT=====
Temperature: 18.0 C
Turbidity: 53.0 NTU
Water Quality Index: 66.5

Water Quality Status: WARNING!

===== WATER SENSOR MONITORING REPORT=====
Temperature: 54.0 C
Turbidity: 65.0 NTU
Water Quality Index: 38.5

Water Quality Status: CRITICAL

===== WATER SENSOR MONITORING REPORT=====
Temperature: 38.0 C
Turbidity: 4.0 NTU
Water Quality Index: 85.0

Water Quality Status: GOOD
```

Question 2 (Transaction Processing and Control Flow)

Link: https://github.com/helen751/Formative1-C_Programming/tree/main/question2_mobile_money

❖ How I Used Conditionals, Loops, Break and Continue

I used **while loop** to keep the main menu running until the user selects Exit. I also used loops in Deposit and Withdrawal, so the user can restart the current transaction or return to the main menu.

I used **conditionals** to validate the user's input and check for errors. The program checks invalid words, invalid numbers, amounts below the required minimum which I specified and withdrawals that are more than the available balance. I added a business rule where the minimum deposit is 100 RWF and the minimum withdrawal is 50 RWF.

I used **switch statement** to run the option selected from the menu.

I used **break** to stop each switch case from moving to the next case. When the user selects Exit, the running condition becomes false and the menu loop stops using break.

I used **continue** when an invalid menu option is entered and when the user chooses to restart the current transaction.

I save every successful transaction in **three arrays**. The arrays store the transaction type, transaction amount and date/time. I used **a loop** to go through the three arrays and print the transaction history and the total transactions and the number of deposits and withdrawals.

❖ MAIN MENU (Good flow, just like a real app)

```
===== MOBILE MONEY TRANSACTION SYSTEM =====
1. Deposit
2. Withdraw
3. Check Balance
4. Transaction Summary
5. Exit
Note: Choose a valid number from 1-4 or 5 to Exit

Choose an action to proceed: 1

Starting Deposit...
```

❖ DEPOSIT PROCESSING (Showing both valid and invalid deposit with error handling)

```
====Welcome, Thadee Gatete!

Enter the deposit amount (at least 100 RWF): -30

Invalid deposit amount, Must be at least 100 RWF

Press y/yes to restart
Press any other character to go back to the main menu
Do you want to perform this transaction again?
```

```
====Welcome, Thadee Gatete!
```

```
Enter the deposit amount (at least 100 RWF): 2000
```

```
Transaction Successful! 2000.00 has been added to your account
```

```
NEW BALANCE: 2000.00
```

```
Press y/yes to restart
```

```
Press any other character to go back to the main menu
```

```
Do you want to perform this transaction again?
```

❖ **WITHDRAWAL PROCESSING (Showing both valid and invalid withdrawal with error handling for insufficient funds)**

```
====Welcome, Thadee Gatete!
```

```
Enter the withdrawal amount (at least 50 RWF): 10000
```

```
Sorry, Insufficient Funds! You only have 2000.00
```

```
Press y/yes to restart
```

```
Press any other character to go back to the main menu
```

```
Do you want to perform this transaction again?
```

```
====Welcome, Thadee Gatete!
```

```
Enter the withdrawal amount (at least 50 RWF): 1000
```

```
Withdrawal Processing... Please take your cash!
```

```
Transaction Successful! You have withdrawn 1000.00 RWF from your account
```

```
NEW BALANCE: 1000.00
```

```
Press y/yes to restart
```

```
Press any other character to go back to the main menu
```

```
Do you want to perform this transaction again? █
```

❖ VIEW TRANSACTION HISTORY

```
=====Transactions History=====
Total Number of Transactions: 3
Deposits (1) | Withdrawals (2)

1. Deposit | 2000.00 RWF | 04-10-2026 16:18:04
2. Withdrawal | 1000.00 RWF | 04-10-2026 16:22:55
3. Withdrawal | 200.00 RWF | 04-10-2026 16:24:36
```

❖ ACCOUNT BALANCE ENQUIRY

```
Hello, Thadee Gatete!
    Your Account Balance is 800.00

===== MOBILE MONEY TRANSACTION SYSTEM =====
1. Deposit
2. Withdraw
3. Check Balance
4. Transaction Summary
5. Exit
    Note: Choose a valid number from 1-4 or 5 to Exit

Choose an action to proceed:
```

Question 3: Functions & Recursive Problem Solving

Link: https://github.com/helen751/Formative1-C_Programming/tree/main/question3_recursive

```
How many Routes do you want to add: rdd
```

```
Error! Enter a valid number.
```

```
How many Routes do you want to add: 34ee
```

```
Error! Enter a valid number.
```

```
How many Routes do you want to add: -1
```

```
Error! Enter a number from 1 and above
```

```
How many Routes do you want to add: 5
```

```
Enter the Distance of Route No 1: |
```

```
How many Routes do you want to add: 5
```

```
Enter the Distance of Route No 1: nn
```

```
Error! Enter a valid number.
```

```
Enter the Distance of Route No 1: 34
```

```
Enter the Distance of Route No 2: 54
```

```
Enter the Distance of Route No 3: 3
```

```
Enter the Distance of Route No 4: 25
```

```
Enter the Distance of Route No 5: 190
```

```
Enter the Distance Limit: 30
```

```
===== DELIVERY DISTANCE ANALYSIS =====
```

```
Distances Given: [34, 54, 3, 25, 190]
```

```
Total Distance: 306km
```

```
Recursive Sum of Distances: 306km
```

```
Average Distance: 61.20km
```

```
Longest Distance: 190km
```

```
Distances Above Set limit of 30km: 3
```

❖ How the program is divided into functions

I divided the program into different functions so that each function has its own responsibility.

The **input_validation()** function checks all the numbers entered in the program. It checks if the user entered a valid integer, if the number was combined with text, and if the number is less than one. It returns *true* when the input is valid and *false* when it is not valid.

The **ask_route_and_limit()** function asks for the number of routes and the distance limit. It uses a while loop and keeps asking until input_validation() returns true. I used one function for both limit and route, encouraging code reuse.

The **add_delivery_routes()** function asks the user to enter each route distance and stores the values inside the integer array. It also uses input_validation() to validate every distance before adding it to the array.

The **total_distance()** function uses a **for loop** to add all the distances.

I also created **recursive_sum()** to calculate the same total using recursion instead of a loop. This allowed me to compare the normal total with the recursive total and confirm that they both returned the same answer.

The **average_distance()** function divides the total distance by the number of routes.

The **longest_distance()** function goes through the array and finds the longest route.

The **no_above_limit()** function counts the number of distances that are greater than the limit entered by the user.

The **print_analysis()** function displays the distances and all the calculated results in a structured way.

The **main()** function controls the program. It asks for the input, creates the distance array, calls the calculation functions, and passes their results to print_analysis().

❖ Function reuse

I reused **input_validation()** for the number of routes, the distance limit, and every route distance. This prevented me from repeating the same validation code in different functions.

I also reused **total_distance()** by calling it to get the total distance and using its returned value as part of the average calculation: **average_distance(total_distance(distances))**

This means I did not have to write the total calculation again inside average_distance().

❖ How the recursive function works

The recursive function is:

```
// A recursive function to perform total using recursive instead of loop
int recursive_sum(int distances[], int size){
    if (size == 0){
        return 0;
    }
    return distances[size - 1] + recursive_sum(distances, size - 1);
}
```

The function receives the distance array and its current size. It adds the last distance to the result of another call with a smaller size.

For example, if the size is 6, it adds the value at index 5, then calls itself with size 5. The next call adds the value at index 4 and calls itself with size 4. This continues until the size becomes zero. This size is the number of array elements.

The base case is:

```
if (size == 0){return 0;}
```

This is where the recursion stops. Without this base case, the function would continue calling itself infinitely.

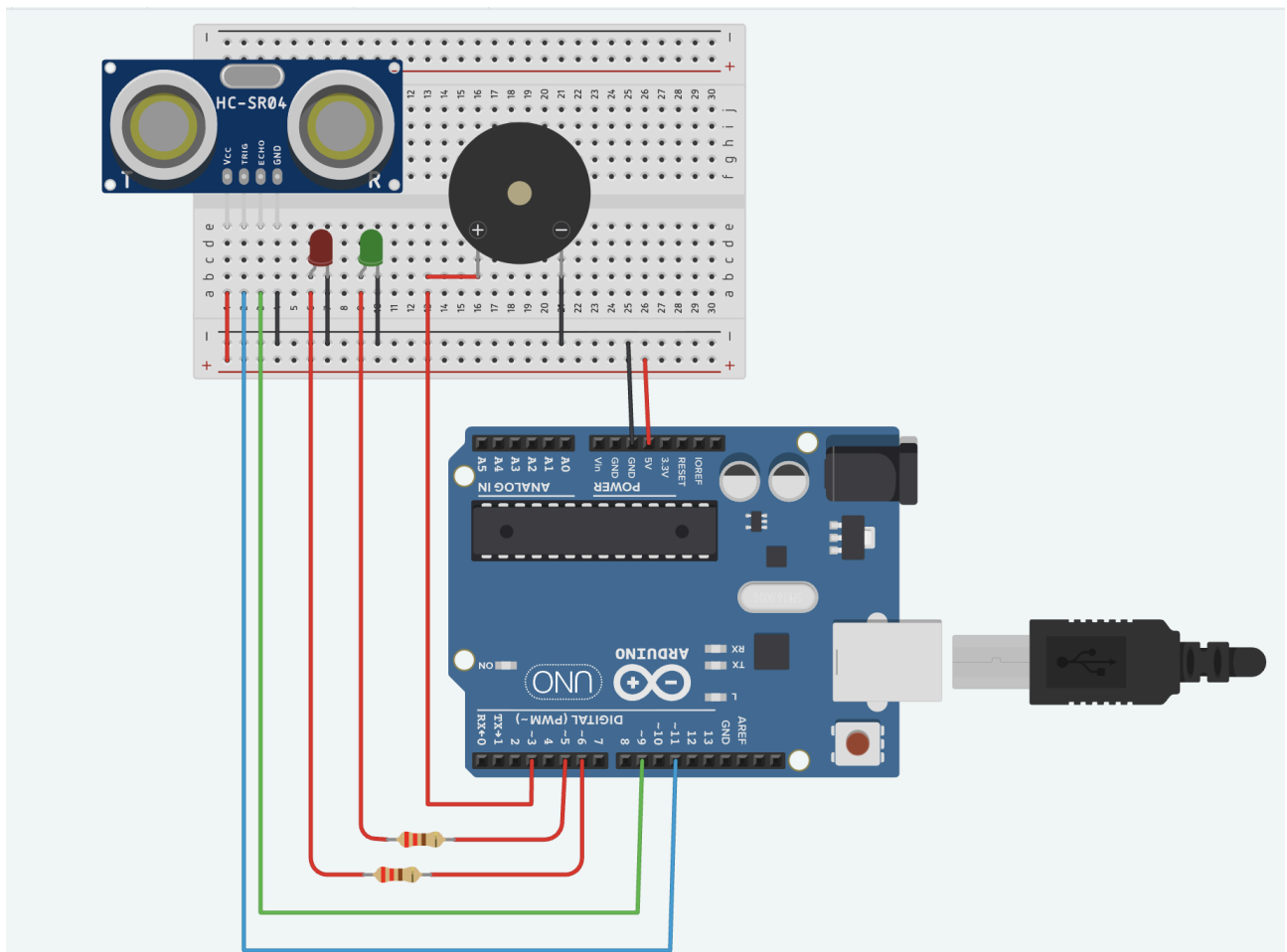
When the base case returns zero, the waiting calls begin returning their answers. Each call adds its distance until all total is returned. This is why the recursive result and the result from `total_distance()` are the same.

❖ Advantages and Disadvantages of the Recursion

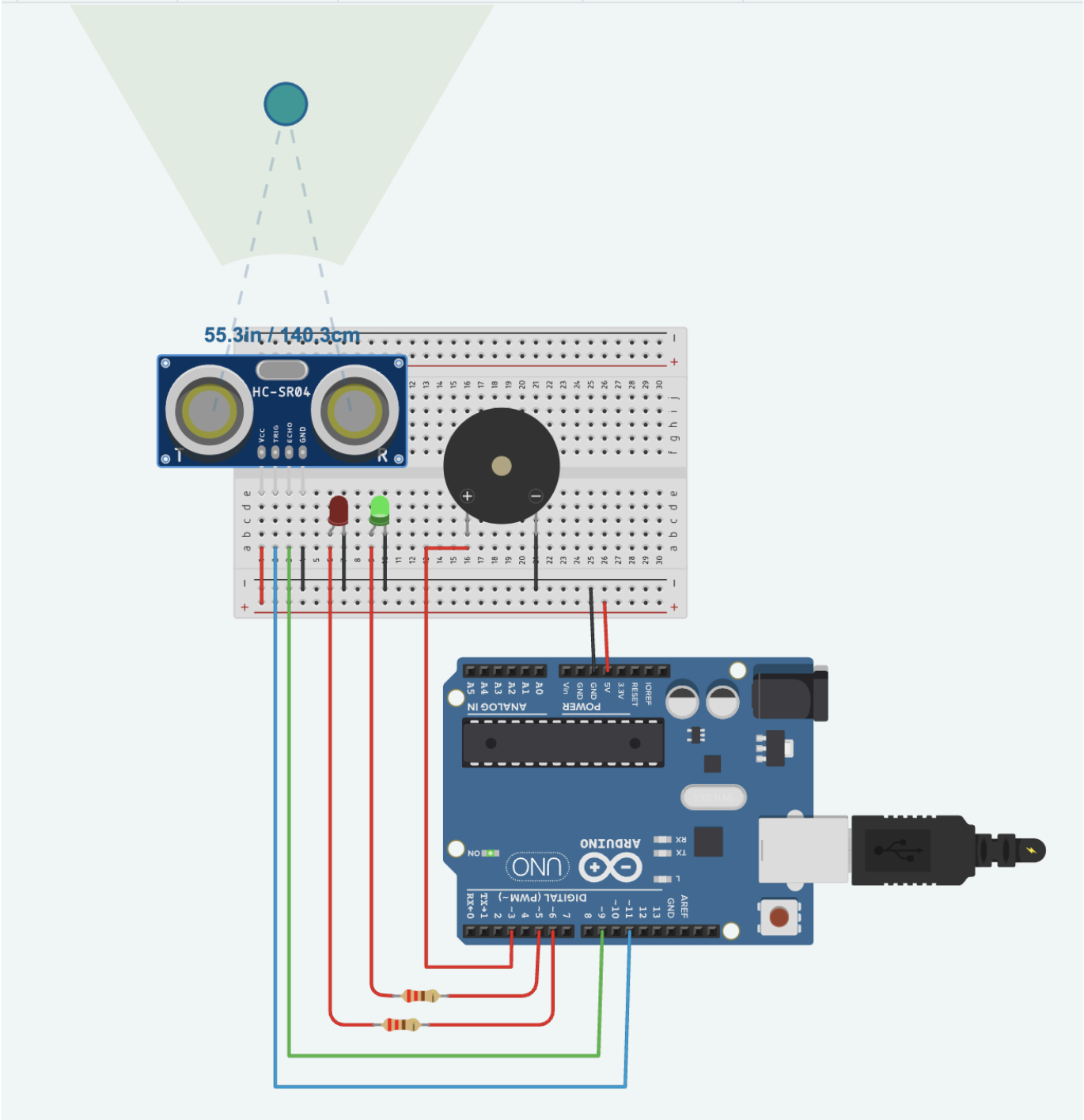
One advantage of recursion here is that it gives a shorter way of expressing the process of adding the array values. One limitation is that every recursive call creates another stack frame and uses more memory. For a very large number of routes, the normal loop would be more memory-efficient.

Question 4: Arduino-Based Smart Parking System

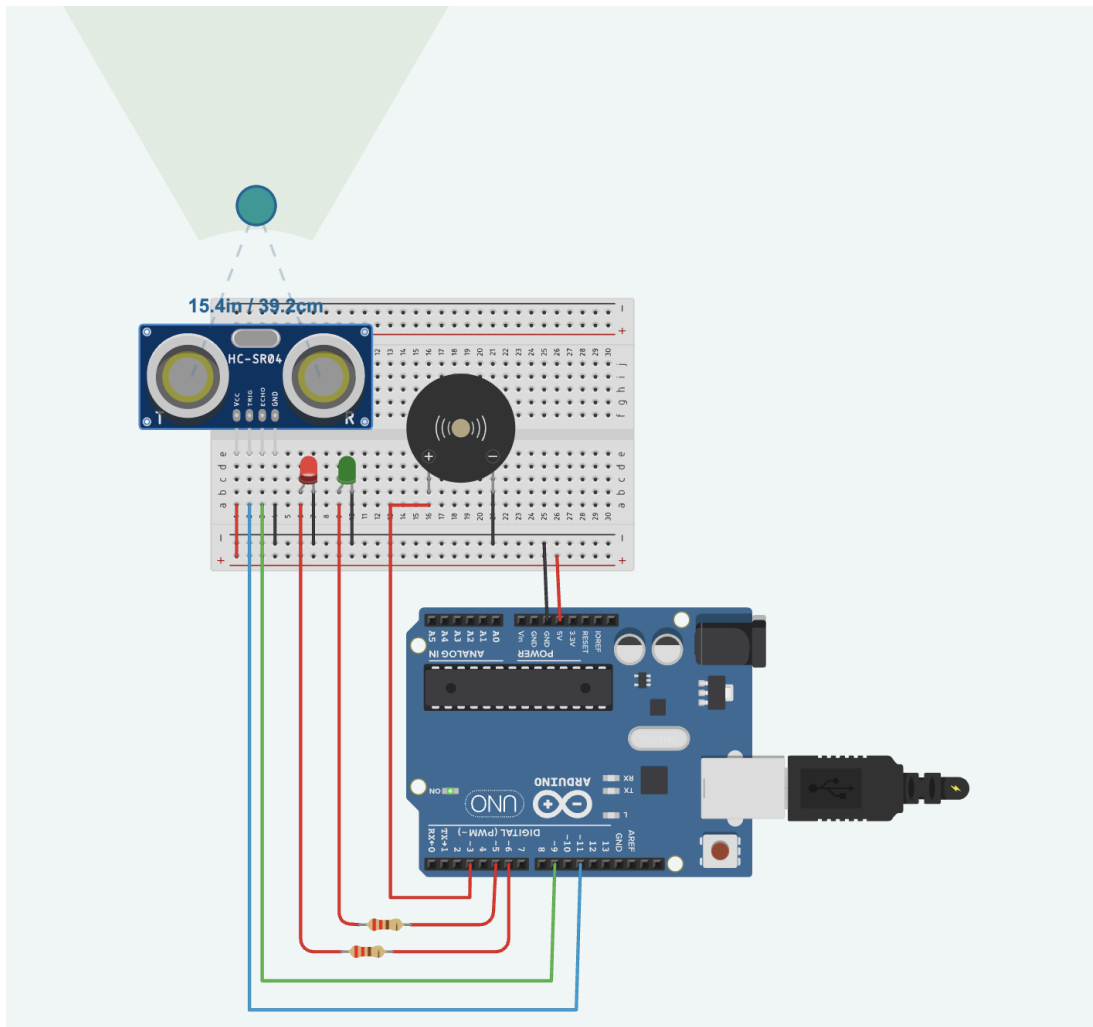
Tinkercad circuit design



Vehicle outside the threshold → Green LED ON, Red LED OFF, Buzzer OFF.



Vehicle within the threshold → Red LED ON, Green LED OFF, Buzzer ON.



❖ Short Explanation

The ultrasonic sensor measures the distance between the sensor and a vehicle.

The Arduino Uno controls the system and processes the reading from the sensor.

The green LED shows that the parking space is available, while the **red LED** shows that it is occupied.

The buzzer gives an alert when a vehicle is detected.

The resistors(220ohms) were used on each **LED** to limit the current flowing through the LEDs, while the **breadboard and jumper** wires connect the components.

The Arduino sends a pulse through the trigger pin of the ultrasonic sensor and measures how long the echo takes to return. It converts this time into distance in centimetres and compares the result with the 50 cm threshold I set.

If the distance is 50 cm or below, the Arduino turns on the red LED and buzzer and turns off the green LED.

If the distance is above 50 cm, it turns on the green LED and turns off the red LED and buzzer.

❖ Block Diagram

